CPSC-406 Report

Andrew Floyd
Chapman University

February 25, 2026

Abstract

Contents

1 Introduction **1**

2 Week by Week **1**
2.1 Week 1 1
2.1.1 Exercise 1: Word Processing with DFAs 1
2.1.2 Exercise 2: Designing DFAs 3

3 Synthesis **5**
3.1 Week 2 5
3.1.1 Exercise 1: Product Automata 5
3.1.2 Exercise 2: More Automata 7

4 Evidence of Participation **9**

5 Conclusion **9**

1 Introduction

2 Week by Week

2.1 Week 1

2.1.1 Exercise 1: Word Processing with DFAs

Notes and Homework:

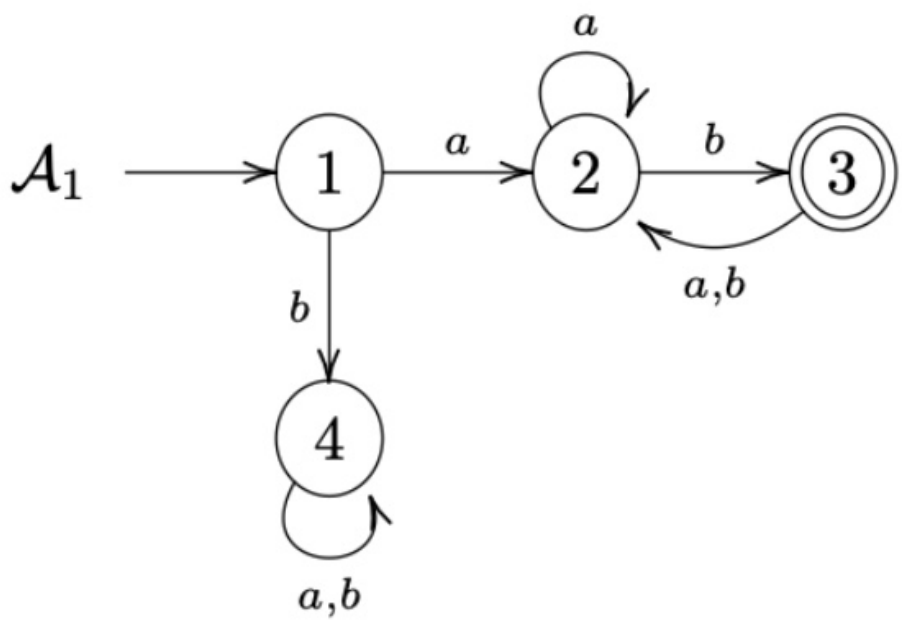

Figure 1: Automata A_1 and A_2

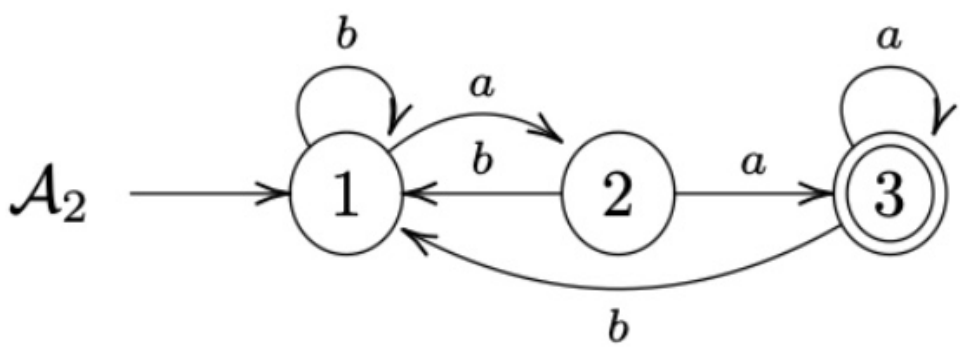

Figure 2: Additional automata reference

Question 1: Which of the following words are accepted/refused by A_1 and A_2, respectively? Complete the table.

w	accepted by A_1?	accepted by A_2?
aaa	no	yes
aab	yes	no
aba	no	no
abb	no	no
baa	no	yes
bab	no	no
bba	no	no
bbb	no	no

Table 1: Acceptance table for A_1 and A_2

Question 2: More generally, can you completely describe the languages $L(A_k)$ accepted by A_k, for $k = 1, 2$?

The language $L(A)$ accepted by this DFA is:

$$L(A) = \{w \in \{a, b\}^* \mid \text{the number of } a\text{'s in } w \text{ is even}\}$$

2.1.2 Exercise 2: Designing DFAs

Design DFAs whose accepted languages are given as follows:

1. All the words that end with ab
2. All the words that contain ab
3. All the words that contain an odd number of a's and an odd number of b's
4. All the words that contain an even number of a's and an odd number of b's
5. All the words such that any three consecutive characters contain at least one a
6. All the words that contain aba

DFA Diagrams:

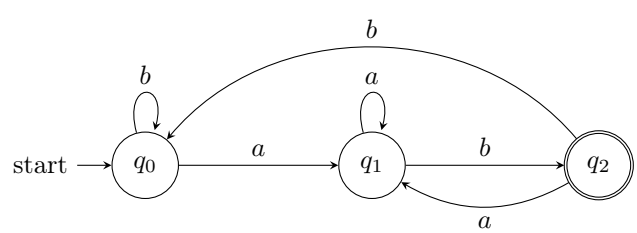

Figure 3: DFA for words ending with ab

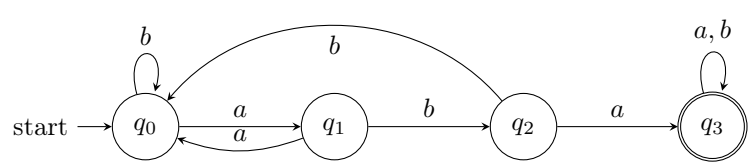

Figure 4: DFA for words containing aba

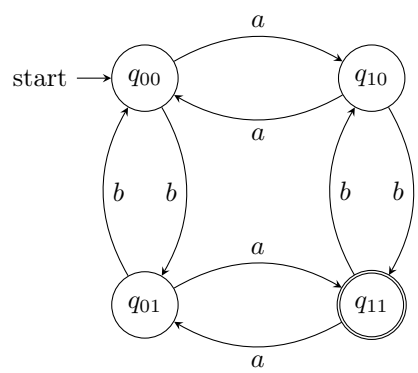

Figure 5: DFA for words with odd number of a's and odd number of b's

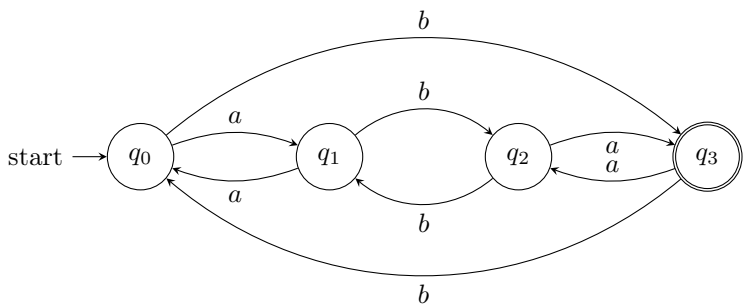

Figure 6: DFA for words with even number of a's and odd number of b's

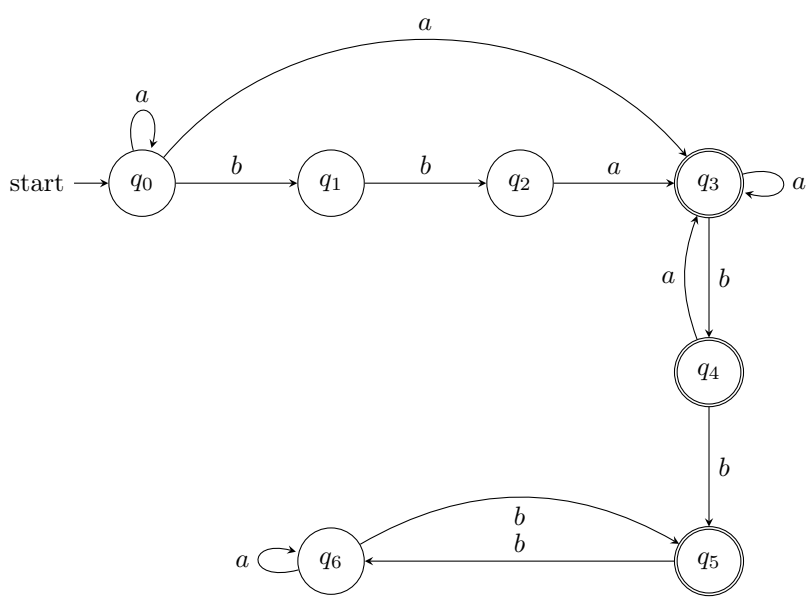

Figure 7: DFA for words where any three consecutive characters contain at least one a

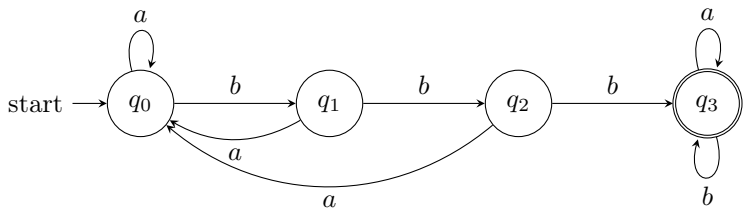

Figure 8: DFA for words containing bbb

What do you notice when comparing the various automata?

When comparing the DFAs designed above, several patterns emerge:

- **State count reflects language complexity:** Simple properties like "ends with ab" require only 3 states, while tracking multiple independent conditions (like counting both a's and b's modulo 2) requires 2^n states where n is the number of independent conditions being tracked.
- **Parity-based languages form grid structures:** DFAs for languages involving "odd/even" counts tend to have states arranged in a grid pattern, where each dimension corresponds to one parity condition. The transitions toggle between states along each axis.
- **Substring containment uses linear chains:** DFAs that check for substring containment (like "contains ab" or "contains bbb") have a linear chain of states tracking progress toward finding the substring, with "reset" transitions back to earlier states when the pattern is broken.
- **Accepting states indicate language structure:** For "contains" languages, once the target is found, we stay in an accepting sink state. For parity languages, specific combinations of parities determine acceptance.
- **Self-loops handle "don't care" inputs:** When an input symbol doesn't affect progress toward acceptance (like a's in "contains bbb" after reaching the accepting state), self-loops keep the automaton in place.

3 Synthesis

3.1 Week 2

3.1.1 Exercise 1: Product Automata

Homework prompt: Work with the two DFAs shown below, describe their languages, and build the product automata needed for intersection and union.

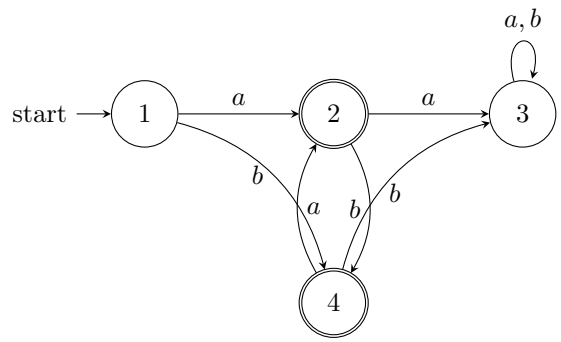

Figure 9: Automaton $\mathcal{A}^{(1)}$

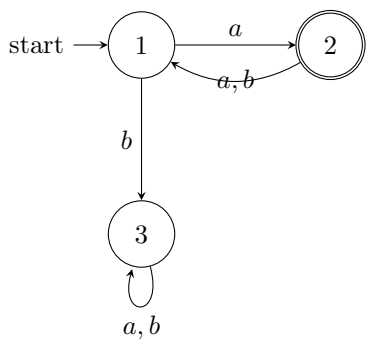

Figure 10: Automaton $\mathcal{A}^{(2)}$

Question 1: Describe $L(\mathcal{A}^{(1)})$ and $L(\mathcal{A}^{(2)})$.

- $L(\mathcal{A}^{(1)})$ is "all strings that stop the moment you land in state 2 or jump from 2 to 4." You must reach 2 by reading an a. You may then loop through "a back to 2, b down to 4" as often as you like, but the word has to end right after that last a into 2 or b into 4.
- $L(\mathcal{A}^{(2)})$ is "all strings that start with a, never fall into state 3, and stay parked in 2." Once you enter 2 on the first a, every later letter must be whatever keeps you there; if you ever take the b to 3 the word is rejected.

Question 2: Construct the intersection automaton $\mathcal{A}$.

The standard product construction uses ordered pairs (p, q) where p is a state of $\mathcal{A}^{(1)}$ and q a state of $\mathcal{A}^{(2)}$. We only draw reachable states.

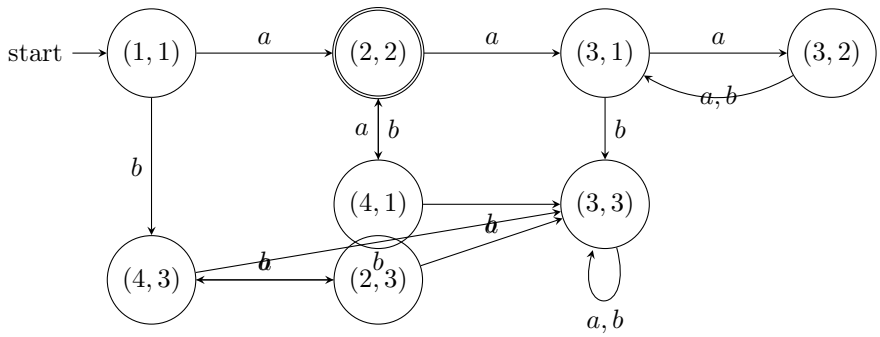

Figure 11: Intersection automaton $\mathcal{A}$

Question 3: Show $L(\mathcal{A}) = L(\mathcal{A}^{(1)}) \cap L(\mathcal{A}^{(2)})$.

The product DFA only has one double-circle state, $(2, 2)$. Getting there means the $\mathcal{A}^{(1)}$ copy is sitting happily in 2 (after possibly bouncing through 4) while the $\mathcal{A}^{(2)}$ copy is also in 2. So a word survives in the product exactly when it makes both original machines happy at the same time, i.e., when it lies in the intersection.

Question 4: Modify $\mathcal{A}$ to obtain $\mathcal{A}'$ with $L(\mathcal{A}') = L(\mathcal{A}^{(1)}) \cup L(\mathcal{A}^{(2)})$.

Keep the same transition structure but change the accepting set to

$$F' = \{(p, q) \mid p \in F_{\mathcal{A}^{(1)}} \text{ or } q \in F_{\mathcal{A}^{(2)}}\}.$$

Now any word accepted by either original automaton drives the product to a state whose pair has at least one accepting component, so the language is the union.

3.1.2 Exercise 2: More Automata

Homework prompt: Repeat the same analysis for the DFAs $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$ below.

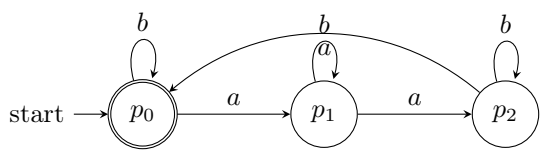

Figure 12: Automaton $\mathcal{B}^{(1)}$

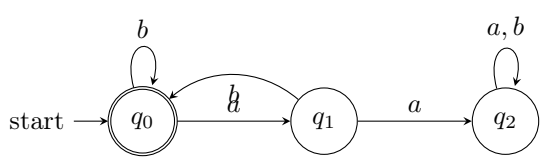

Figure 13: Automaton $\mathcal{B}^{(2)}$

Question 1: Describe $L(\mathcal{B}^{(1)})$ and $L(\mathcal{B}^{(2)})$.

- $L(\mathcal{B}^{(1)})$ = "count a's in groups of three and ignore b's." The machine walks through $p_0 \to p_1 \to p_2 \to p_0$ every time an a shows up, so you only end in the accepting home state p_0 when the total number of a's is a multiple of three.
- $L(\mathcal{B}^{(2)})$ = "never write two a's back-to-back." You can stay in q_0 on b's, hop to q_1 on a single a, but a second immediate a throws you into the sink q_2. As long as every a is separated by at least one b, you return to q_0 and the word is accepted.

Question 2: Construct the intersection automaton $\mathcal{B}$.

The full product has nine reachable states. Starting from (p_0, q_0) we obtain the transition graph shown below. Because only p_0 and q_0 are accepting in their respective machines, the sole accepting pair is (p_0, q_0).

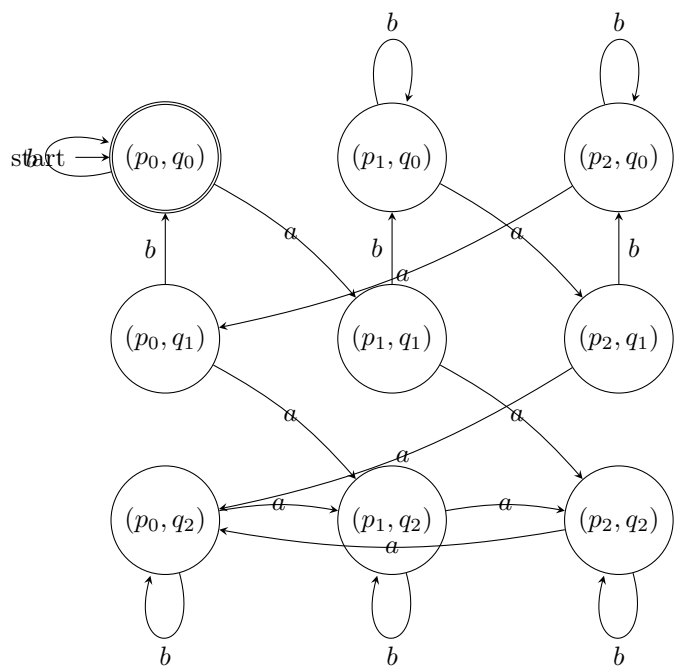

Figure 14: Intersection automaton $\mathcal{B}$

Question 3: Why does $L(\mathcal{B}) = L(\mathcal{B}^{(1)}) \cap L(\mathcal{B}^{(2)})$?

Each arrow in the product just mirrors the arrows in the two base DFAs. The only double-circle state is (p_0, q_0), so the combined machine says "yes" exactly when both halves also say "yes" at the end of the word. That is the definition of the intersection.

Question 4: Obtaining the union language.

As in Exercise 1, we can apply De Morgan's law. Complement each DFA by flipping accepting/non-accepting states, build the product, and complement again. Equivalently, start from the product above and simply mark as accepting every pair where at least one component is accepting. Either construction yields an automaton $\mathcal{B}'$ whose language is $L(\mathcal{B}^{(1)}) \cup L(\mathcal{B}^{(2)})$.

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, Title, Publisher, Year.